

1. JIT Overhead at Runtime

JIT means **Just-In-Time compilation**. In .NET, the C# compiler first converts C# code into **Intermediate Language (IL)**. When the program runs, JIT converts the needed IL into native machine code that the CPU can execute.

Simple flow:

```
C# Code → C# Compiler → IL → JIT Compiler → Native Machine Code → CPU
```

Why is there an overhead?

Some compilation happens while the program is running. For example, when a method is used for the first time, JIT may compile it before it can run. This can add some startup or first-use time.

How is it reduced?

- **JIT on demand:** it does not compile every method at the beginning.
- **ReadyToRun:** some code is compiled before the application starts, so JIT has less work.
- **Tiered compilation:** .NET can start with quickly generated code and later optimize frequently used methods.
- **Native AOT:** the application is compiled to native code before running, so it does not use JIT at runtime.

JIT gives flexibility and runtime optimization, but it can add runtime work. Modern .NET uses different techniques to reduce this cost.

2. Hello World using VS Code

To create a simple .NET console application using VS Code, I need the **.NET SDK** and VS Code.

Steps:

- Install the .NET SDK and Visual Studio Code.
- Open VS Code and its terminal.
- Create a console project:

```
dotnet new console -n HelloWorld
```

- Enter the project folder:

```
cd HelloWorld
```

- Run the application:

```
dotnet run
```

The program prints **Hello, World!** in the terminal.

Example code:

```
Console.WriteLine("Hello, World!");
```

Console is used to work with the console, and **WriteLine** prints text to the screen.

.NET Versions

.NET started with the original **.NET Framework**. Later, Microsoft introduced **.NET Core** as a cross-platform and open-source implementation. Starting with **.NET 5**, Microsoft unified the modern platform and dropped “Core” from the name.

Important versions:

Version	Simple meaning
.NET Core 1.0 → 3.1	Earlier cross-platform .NET implementation.
.NET 5	Unified modern .NET line started; “Core” removed from the name.
.NET 6	Modern .NET; LTS.
.NET 7	Modern .NET; STS.
.NET 8	Modern .NET; LTS.
.NET 9	Modern .NET; STS.
.NET 10	Current modern release in August 2026; LTS.

The important idea is that **.NET Core** was the earlier cross-platform line. From **.NET 5**, it became simply **.NET**.

Namespace

A **namespace** is a way to organize related classes, interfaces, and other types in C#. It also helps avoid naming conflicts in large applications.

Example:

```
namespace MyApp.Services;

class UserService
{
    // code here
}
```

Here, **UserService** belongs to the **MyApp.Services** namespace.

What does using do?

The **using** keyword lets me use types from a namespace without writing the full namespace every time.

```
using System;

Console.WriteLine("Hello");
```

Without using System, I could write **System.Console.WriteLine()**. It mainly makes the code shorter and easier to read.

.NET Core

.NET Core was a newer version of .NET designed to be **cross-platform** and **open source**. Unlike the original .NET Framework, which was mainly focused on Windows, .NET Core was designed to run on Windows, Linux, and macOS.

Why was .NET Core important?

- It could run on more than one operating system.
- It was open source.

- It was designed for modern web, cloud, console, and other applications.
- It was modular, so applications could use the libraries they needed.

After .NET Core 3.1, Microsoft continued the platform with .NET 5. The word “Core” was removed because the modern platform became the main unified .NET product.

Solution

A **solution** is a container used in Visual Studio to organize one or more related projects. For example, one solution can contain a Web API project, a class library project, and a test project.

Example:

```
MyCompany.sln
■
■■■ MyCompany.API
■■■ MyCompany.Core
■■■ MyCompany.Tests
```

Here, **MyCompany.sln** is the solution, and inside it there are three projects. Each project has its own code and configuration.

Project vs Solution

Project	Solution
Contains files needed for one application or library.	Organizes one or more related projects.
Example: API project.	Example: API + Core + Tests.
C# projects commonly use .csproj.	Visual Studio solutions commonly use .sln.